```
def a_method
  return yield if block_given?
  'no block'
end

a_method                          # => "no block"
a_method { "here's a block!" }    # => "here's a block!"
```

If you use yield when block_given? is false, you'll get a runtime error.

Now you can apply what you know about blocks to a real-life scenario.

Quiz: Ruby#

Where you're challenged to do something useful with blocks.

Bill shares a little secret: "You know, a few years ago I was making a living out of writing C# code. I must admit that C# did have a few nice features. Let me show you one of those."

The using Keyword

Imagine that you're writing a C# program that connects to a remote server and you have an object that represents the connection:

```
RemoteConnection conn = new RemoteConnection("my_server");
String stuff = conn.ReadStuff();
conn.Dispose();    // close the connection to avoid a leak
```

This code correctly disposes of the connection after using it. However, it doesn't deal with exceptions. If ReadStuff throws an exception, then the last line is never executed, and conn is never disposed of. What the code *should* do is manage exceptions, disposing of the connection regardless of whether an exception is thrown. C# provides a keyword named using that goes through the whole process for you:

```
RemoteConnection conn = new RemoteConnection("some_remote_server");
using (conn)
{
    conn.ReadData();
    DoMoreStuff();
}
```

The using keyword expects that conn has a method named Dispose. This method is called automatically after the code in the curly braces, regardless of whether an exception is thrown.